
Owner's Manual: Autonomous Crop-Health Monitoring System

CropWatcher - Crazyflie-based indoor scouting, telemetry, mapping, and waypoint operation

This manual explains how to safely operate, configure, troubleshoot, and maintain the CropWatcher autonomous crop-health monitoring system. It has been revised to include the final waypoint system, the front-end Flight Control workflow, organized telemetry logging, and the actual flight scripts used by the system.

1. Introduction

1.1 System Overview

The Autonomous Crop-Health Monitoring System is an indoor greenhouse scouting platform built around a modified Crazyflie 2.1 drone, Lighthouse positioning, onboard environmental sensing, and a browser-based CropWatcher dashboard. The system collects environmental data during flight, maps that data to greenhouse zones, and supports machine-learning-assisted crop-health analysis so operators can identify problem areas earlier than manual scouting alone.

The drone is designed for scouting only. It does not spray, treat, trim, or physically contact plants. Its main purpose is to collect repeatable environmental and spatial data that can be reviewed live or after the mission.

1.2 Goal: Maximizing Yield Through Earlier Detection

Small environmental changes in a greenhouse can become crop stress before symptoms are obvious to a person walking the rows. CropWatcher helps growers collect repeatable data more often, compare runs over time, and focus manual inspection on the zones most likely to need attention.

- **Consistency:** The same mission type can be repeated across days for comparable results.
- **Objectivity:** Sensor readings and model scores reduce guesswork in early stress detection.
- **Speed:** The drone can collect environmental samples and position data quickly across a defined flight area.
- **Traceability:** Timestamped CSV logs allow each flight to be reviewed, graphed, and compared.

1.3 Key Features

- **Autonomous and semi-autonomous flight modes:** hover testing, lawnmower scanning, and waypoint missions.
- **Lighthouse-based localization:** stateEstimate.x/y/z coordinates are used for position hold, flight-path visualization, and waypoint navigation.
- **Temperature correction:** raw BMP388 temperature is corrected using a thermal model that accounts for heat effects and flight state.
- **Pressure and air-density logging:** station pressure, sea-level pressure estimate, altitude, and air density are saved with each data row.

- Live dashboard integration: telemetry can be published to the local API server and displayed in the CropWatcher interface.
- Organized flight logs: flight CSVs are saved into date-based folders with timestamped filenames to prevent overwrites.
- Waypoint system: operators can define custom scouting paths using a 5 ft x 5 ft clickable map or the dashboard Layout Editor.

1.4 What's in the Box

Component	Quantity	Description
Crazyflie 2.1 drone	1	Assembled micro-drone platform with expansion decks used for the project.
Lighthouse deck	1	Provides indoor positioning data from Lighthouse base stations.
BMP388 / barometer sensor support	1	Used for temperature, pressure, altitude, and air-density calculations.
AI Deck	1	Mounted expansion deck. Camera-related functions should be treated as optional/diagnostic unless camera streaming is confirmed in your build.
LiPo batteries	2	Flight batteries for the Crazyflie.
Battery charger	1	USB-powered charger for flight batteries.
Lighthouse base stations	2	V2 units for indoor positioning.
Crazyradio PA	1	USB radio used for communication between the host computer and Crazyflie.
MicroUSB cable	1	Used for charging, flashing, and USB connections.
Spare propellers	1 set	Clockwise and counter-clockwise replacement propellers.
Extra motor arms	2	Replacement frame arms.
Extra motor	1	Replacement motor.

2. Safety and Compliance

2.1 General Safety Rules

- Indoor use only: The Crazyflie is lightweight and should not be operated outdoors or in windy areas.
- Operator supervision required: A trained operator must remain present for every flight.
- Clear the flight zone: Remove loose wires, hanging plastic, strings, tools, ladders, and people from the flight path before takeoff.
- Do not fly over people: Keep the flight path clear of staff and visitors.
- Keep the drone away from water: Do not allow the drone, battery, or electronics to contact nutrient solution, irrigation water, or wet surfaces.

- Respect facility rules: Only fly in approved indoor areas and follow any greenhouse-specific safety procedures.

2.2 Propeller and Battery Safety

- Never touch the drone while the propellers are spinning.
- Inspect propellers before every flight for cracks, nicks, bent blades, or stress whitening.
- Use the correct propeller orientation. Incorrect CW/CCW placement can cause immediate tumble on takeoff.
- Do not use swollen, punctured, overheated, or physically damaged LiPo batteries.
- Disconnect the battery after a crash or after the mission is complete.

2.3 Automated and Operator-Controlled Safety Features

- Geofence limits: The dashboard and waypoint workflow reject coordinates outside configured X/Y boundaries.
- Lighthouse monitoring: If the position estimate is unstable or missing, do not launch. Recalibrate before flying.
- Stop controls: Use the dashboard Stop Script button or press Ctrl+C in the flight-script terminal to abort.
- Emergency landing: Flight scripts attempt to land on exceptions instead of leaving the drone running in the air.
- Dry-run mode: The standalone waypoint planner supports a dry-run option so the operator can verify coordinates before a real flight.

2.4 Emergency Procedures

- Immediate abort from terminal: Press Ctrl+C in the active flight terminal. The script should begin landing.
- Immediate abort from dashboard: In the Flight Control tab, press Stop Script if a flight script is running.
- After a crash: Wait for motors to stop, disconnect the battery, inspect propellers and arms, and do not relaunch until the cause is understood.
- If the drone becomes unresponsive: Use the Recover button in cfclient when appropriate, then power-cycle the drone.

3. Software Environment Installation

3.1 Host Requirements

- Operating system: Windows 10/11 or macOS.
- Python: Python 3.11 is recommended for this project environment.
- USB: One available USB port for Crazyradio PA.
- Browser: A modern browser for the CropWatcher dashboard.
- Optional: Docker Desktop if firmware or AI Deck Wi-Fi settings need to be rebuilt and flashed.

3.2 Install Python 3.11

Windows

1. Download the Python 3.11 installer from python.org.
2. Run the installer and check Add Python to PATH before selecting Install Now.

3. Open PowerShell and verify the install:

```
python --version
```

macOS

4. Install Homebrew if it is not already installed.

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

5. Install Python 3.11 and libusb:

```
brew install python@3.11 libusb
```

6. Verify the install:

```
python3.11 --version
```

3.3 Create the Project Workspace and Virtual Environment

7. Create a folder named greenhouse_drone in your Documents folder or clone the final repository into your preferred project folder.

8. Open PowerShell or Terminal and move into that folder.

```
cd Documents/greenhouse_drone
```

9. Create a virtual environment.

```
python -m venv venv # Windows  
python3.11 -m venv venv # macOS
```

10. Activate the virtual environment.

```
.\venv\Scripts\activate # Windows  
source venv/bin/activate # macOS
```

3.4 Install Dependencies

With the virtual environment active, install the Crazyflie client/library and common project dependencies. The exact dependency list depends on the repository, but the following are the core packages used by the project scripts.

```
pip install --upgrade pip  
pip install cflib cfcclient pandas matplotlib
```

3.5 Windows Crazyradio Driver Setup

11. Download Zadig from zadig.akeo.ie.

12. Plug in the Crazyradio PA.

13. Open Zadig and choose Options > List All Devices.

14. Select Crazyradio PA from the device dropdown.

15. Select libusb-win32 on the right side and click Replace Driver.

3.6 Initial Live Connection Test

16. Insert a charged battery and power on the Crazyflie.

17. Launch cfcclient from the activated virtual environment.

```
cfcclient
```

18. Confirm the Crazyradio PA appears in the upper-left connection dropdown.

19. Connect to the drone and verify the artificial horizon responds when you carefully tilt the drone by hand.

20. Open the positioning/Lighthouse view and confirm X, Y, and Z update when the drone is moved.

4. Hardware Setup and Calibration

4.1 Battery Installation

Slide the LiPo battery into the cavity between the AI Deck and the mainboard. Connect the white battery plug to the Crazyflie power port and confirm the wires are tucked away from the propellers.

4.2 Lighthouse Base Station Setup

21. Mount two Lighthouse base stations in opposite corners of the flight area, ideally around 2 meters high.
22. Point both stations toward the origin or center of the flight volume.
23. Connect each base station to the computer and use `cfclient` to assign separate channels, commonly Channel 1 and Channel 2.
24. Make sure the stations have a clear line of sight to the drone throughout the planned path.

4.3 Lighthouse Geometry Calibration

25. Open `cfclient` and navigate to the Lighthouse Positioning tab.
26. Place the drone at the chosen origin and keep it still.
27. Use Manage Geometry > Estimate Geometry and follow the prompts.
28. Return the drone to the origin and confirm X, Y, and Z are close to 0, 0, 0.
29. Repeat calibration any time a base station is moved, rotated, or bumped.

4.4 Stability and Performance Testing

Hover Test

Use `hover_test.py` to confirm the drone can take off, hold its X/Y position, and land safely before attempting mapping or waypoint missions.

```
python hover_test.py
```

- Run at low altitude first, such as 1.5-2 ft, then test at the expected mission height.
- If the drone drifts, wobbles, or fights position hold, stop and recalibrate Lighthouse before continuing.

Lawnmower Flight Test

Use `lawnmower_flight.py` for structured coverage of a square area. It scans in a serpentine pattern and collects the same sensor data used by the dashboard.

```
python lawnmower_flight.py
```

Waypoint Flight Test

Use `waypoint_flight.py` to test custom point-by-point routes. Start in dry-run mode to verify points before enabling a real flight.

```
python waypoint_flight.py
```

5. Deployment and Operating Procedures

5.1 Daily Startup

30. Walk the greenhouse rows and remove temporary obstacles from the flight path.
31. Power both Lighthouse base stations and confirm normal status lights.
32. Inspect the drone, propellers, arms, battery, and expansion decks.
33. Start the host computer and open a terminal in the project folder.

34. Activate the virtual environment.

```
source venv/bin/activate      # macOS
.\venv\Scripts\activate      # Windows
```

35. Start the local API/dashboard server.

```
python api_server_final.py
```

36. Open the dashboard in a browser. The server hosts the interface from the local project folder.

```
http://127.0.0.1:5001
```

5.2 Operator Pre-Flight Checklist

- [] Propellers are attached, undamaged, and installed in the correct CW/CCW positions.
- [] Battery is secure and the connector is fully seated.
- [] Flight area is clear of people, tools, hanging plastic, wires, and water hazards.
- [] Crazyradio PA is connected and cflight can communicate with the drone.
- [] Lighthouse positioning is green/stable, and X/Y/Z values respond correctly.
- [] The correct Crazyflie URI is selected in the dashboard or script.
- [] The intended mission type has been reviewed: hover, lawnmower, or waypoint.
- [] Ambient temperature entry is ready in the desired unit, such as 74F or 22C.
- [] The operator knows where the Stop Script button and terminal Ctrl+C abort are located.

5.3 Mission Types

Mission Type	Script	Best Use	Main Inputs
Hover test	hover_test.py	Quick stability, Lighthouse, and position-hold test before real missions.	Hover altitude in feet.
Lawnmower scan	lawnmower_flight.py	Repeatable coverage over a square area using serpentine rows.	Ambient temperature; configured square size, step size, altitude layers.
Waypoint mission	waypoint_flight.py	Custom path selected by the operator using a 5 ft x 5 ft clickable map or dashboard waypoint tools.	Crazyflie URI, ambient temperature, flight height, waypoint list.

5.4 Starting a Mission from the Dashboard

37. Start api_server_final.py in the same folder as mainFrontEnd_final.html, waypoint_flight.py, hover_test.py, and lawnmower_flight.py.

38. Open http://127.0.0.1:5001 in the browser.

39. Open the Flight Control tab.

40. Confirm Flight Control Online is displayed.

41. Enter the Crazyflie URI, such as radio://0/80/2M/E7E7E7E7E7.

42. Enter the ambient temperature in the same unit you want stored in the CSV, such as 74F or 22C.

43. Select the mission button: Launch waypoint_flight.py, Run hover_test.py, or Run lawnmower_flight.py.

44. Watch the Flight Launcher Output panel for script messages and errors.

45. Use Stop Script if the drone becomes unstable or the wrong script was launched.

5.5 Post-Mission Procedure

46. Wait for the motors to stop completely before approaching the drone.
47. Disconnect the battery and allow motors/electronics to cool if needed.
48. Check the dashboard for newly loaded telemetry and graphs.
49. Confirm the CSV was saved in `organized_flights/YYYY-MM-DD/`.
50. Inspect the drone for damaged props, loose decks, or bent motor arms before the next flight.

6. Waypoint System

6.1 What the Waypoint System Does

The waypoint system allows the operator to define a custom scouting path instead of flying only a fixed lawnmower pattern. Each waypoint is a target X/Y/Z position. The drone takes off, travels to each point in sequence, collects telemetry during the mission, and lands when the route is complete or aborted.

The project includes two related waypoint workflows: a standalone 5 ft x 5 ft clickable Tkinter planner in `waypoint_flight.py`, and the dashboard Layout Editor / waypoint mission tools that define zones, save mission files, and queue paths through the API server.

6.2 Coordinate Model

- Standalone waypoint planner: The map is 5 ft x 5 ft. The center of the canvas is the drone starting position. Clicked points are stored in feet relative to the starting point and converted to meters before flight.
- Dashboard Layout Editor: The greenhouse grid is split into labeled zones such as A1, A2, B1, and so on. Clicking a zone creates a waypoint at that zone center using the configured geofence bounds.
- Flight execution: The flight script captures the current Lighthouse state estimate and computes absolute `go_to` targets from the starting position plus each waypoint offset.
- Altitude: Standalone waypoint settings are entered in feet. Dashboard waypoint default hover height is shown in meters.

6.3 Standalone 5 ft x 5 ft Waypoint Planner

Use this workflow when you want a simple custom path that is centered on the drone's current starting location.

51. Make sure Lighthouse is calibrated and the drone is placed at the desired starting point.
52. Activate the virtual environment and run the waypoint planner.
`python waypoint_flight.py`
53. Confirm the Crazyflie URI is correct.
54. Enter the flight height in feet.
55. Enter the ambient room temperature, such as 74F or 22C. This value is passed into the same sensor logging used by `lawnmower_flight.py`.
56. Leave Dry run enabled for the first attempt. Click the desired points on the map and verify the converted waypoint list.
57. Use Undo Point or Clear if the route needs to be changed.
58. When the route is correct and the flight area is clear, disable Dry run and launch the flight.
59. Monitor the terminal and be ready to abort with `Ctrl+C`.

Recommended Standalone Waypoint Settings

Setting	Recommended Starting Value	Purpose
---------	----------------------------	---------

Flight height	1.5-2.0 ft for early tests	Keeps the drone low while confirming route behavior.
go_to duration	3.0 seconds	Controls how long each movement command takes. Increase for smoother/slower movement.
go_to wait window	3.5 seconds	Wait time allowed for each waypoint.
Return to start before landing	Enabled	Brings the drone back near the origin before landing.
Dry run	Enabled for first route check	Verifies coordinate conversion without spinning motors.
Static map check	Enabled if map files are available	Rejects paths that cross obstacles or leave the defined operational area.

6.4 Dashboard Layout Editor Waypoint Workflow

Use this workflow when you want waypoint selection to be part of the CropWatcher dashboard. The Layout Editor lets the operator define zones, choose waypoints, review the sequence, save mission files, and execute or cancel a mission.

60. Start `api_server_final.py` and open the dashboard.

```
python api_server_final.py
http://127.0.0.1:5001
```

61. Open the Layout Editor tab.

62. Review the greenhouse grid dimensions and geofence limits.

63. Click zones to add waypoints. Click a selected zone again or use the remove control to remove it.

64. Review the Waypoint Sequence list. Confirm the order, X/Y/Z values, and estimated flight time.

65. Set the default hover height. Stay conservative during early testing.

66. Use Export Waypoints if you need a CSV copy of the planned path.

67. Use Execute Mission only after the route, geofence, drone positioning, and flight area have been checked.

68. Use Cancel if the mission should be stopped before completion.

6.5 Waypoint Mission Files

When a dashboard mission is created, the API server writes a timestamped mission text file and updates the latest mission file. These files make the route human-readable and easy for the backend/bridge to pick up:

```
mission_paths/mission_YYYYMMDD_HHMMSS.txt
mission_paths/latest_mission.txt
```

The mission text format lists each waypoint with sequence number, zone label, X meters, Y meters, and Z meters. This gives the operator a simple record of what path was requested.

6.6 Waypoint Safety Rules

- Always run `hover_test.py` successfully before running a waypoint mission in a new setup.
- Run the standalone waypoint planner in dry-run mode before disabling dry run for real flight.
- Keep waypoints inside the Lighthouse coverage area and away from crop canopies, strings, rack edges, and water.

- Use slow go_to durations if the drone is close to obstacles or if the route has sharp turns.
- Never assume the dashboard route is safe just because the software accepted it. The operator must visually inspect the real space.
- If the drone misses a waypoint, drifts, or moves unpredictably, stop the script and recalibrate Lighthouse.

6.7 Troubleshooting Waypoint Missions

Issue	Likely Cause	Fix
Waypoint planner opens but drone does not fly	Dry run is still enabled or Crazyflie URI is wrong.	Disable dry run only after checking the route. Confirm the URI and Crazyradio connection.
No position estimate received	Lighthouse or Kalman estimator is not providing stateEstimate.x/y/z.	Reopen cfclient, check Lighthouse geometry, reset estimator, and verify position data.
Waypoint rejected	Static map check or geofence detected an unsafe point/path.	Move the waypoint, update the map/geofence, or disable map check only if the real area is known to be clear.
Drone flies relative to the wrong origin	The drone was moved after the route was planned or stale estimate data was used.	Place the drone at the desired start position, keep it still, reset estimator, and relaunch.
Script blocks at ambient temperature prompt	The script is waiting for a temperature value.	Enter a value like 74F or launch through the dashboard, which passes the ambient value to the script.
Dashboard says bridge/API not connected	api_server_final.py is not running or the page cannot reach localhost:5001.	Start the API server in the project folder and refresh the dashboard.

7. System Reconfiguration and Network Management

7.1 Relocating to a New Greenhouse Zone

69. Move Lighthouse base stations to the new zone using the same height/orientation guidance from Section 4.
70. Update dashboard geofence limits to match the new safe area.
71. If using the standalone waypoint planner with a static map, confirm the YAML/PGM map files match the new space.
72. Re-run Lighthouse geometry estimation any time a base station is moved.
73. Run hover_test.py before running lawnmower or waypoint missions.

7.2 Changing AI Deck Wireless Network Settings

If AI Deck Wi-Fi settings need to be changed, rebuild and flash the firmware using the Bitcraze Docker Toolbelt. This is an advanced procedure and should only be done by a technical operator.

74. Install Docker Desktop for the host platform.
75. From the crazyflie-firmware folder, run menu configuration.

```
docker run --rm -it -v ${PWD}:/module bitcraze/toolbelt make nconfig
```

76. Navigate to the AI Deck support/Wi-Fi configuration options, set the SSID and password, save, and exit.
77. Build the firmware.

```
docker run --rm -it -v ${PWD}:/module bitcraze/toolbelt make
```
78. Flash the main STM32 firmware if needed.

```
python3 -m cfloader flash build/cf2.bin stm32-fw
```
79. Flash the AI Deck GAP8 firmware if needed.

```
python3 -m cfloader flash [path_to_gap8_img] deck-bcAI:gap8-fw
```

8. Data Collection and Processing

8.1 Sensor Suite and Model Inputs

The system focuses on environmental telemetry and position-tagged records. The machine-learning/dashboard workflow can use available sensor readings and optional imagery/canopy features when present. Camera features should be treated as optional unless your current hardware and firmware are confirmed to stream images reliably.

- Temperature: raw BMP388 temperature plus corrected temperature from the thermal correction engine.
- Atmospheric pressure: station pressure and calculated sea-level pressure estimate.
- Altitude: relative altitude in meters and feet.
- Air density: calculated from pressure and corrected temperature.
- Battery voltage and thrust: used for flight monitoring and contextual interpretation.
- Lighthouse X/Y/Z: used for flight path plotting, zone mapping, and waypoint execution.

8.2 Ambient Temperature Prompt and Unit Selection

At mission launch, enter an ambient temperature reference such as 74F or 22C. The unit letter controls how temperature values are saved in that mission CSV. If F is entered, temperature columns are saved as Fahrenheit. If C is entered, temperature columns are saved as Celsius.

Accuracy note: Use a trustworthy room thermometer or greenhouse sensor near the start position. The ambient value affects the corrected temperature calculations and should represent the starting environment.

8.3 Organized CSV Storage

Flight logs are saved into a date-based folder under `organized_flights`. This avoids overwriting earlier runs and makes dashboard historical comparison easier.

```
organized_flights/YYYY-MM-DD/drone_mapping_data_YYYY-MM-DD_HH-MM-SS.csv
```

The API server loads the latest dated CSV first, then falls back to the legacy `drone_mapping_data.csv` if no organized file exists.

8.4 Recorded CSV Parameters

Column / Group	Meaning
index	Sequential reading number.
datetime_et	Timestamp when the data row was captured.
state, mode, event	Current flight/sensor state and thermal correction event.

roc_F_per_s or roc_C_per_s	Rate of change for the smoothed temperature signal.
raw_temp_F/C	Raw sensor temperature in the unit selected at launch.
expected_raw_F/C	Expected raw temperature based on the ambient estimate and heat offset.
deviation_F/C	Difference between measured and expected raw temperature.
corr_temp_F/C	Corrected temperature used for analysis.
thermal_offset_F/C	Estimated internal heat offset.
ambient_est_F/C	Current ambient estimate used by the correction engine.
rel_alt_m, rel_alt_ft	Relative altitude.
air_density_kg_m3	Calculated dry-air density.
station_pressure_hPa	Measured local station pressure.
sea_level_pressure_QFF_hPa	Estimated sea-level pressure.
battery_V	Drone battery voltage.
thrust	Motor output level.
lh_x_m, lh_y_m, lh_z_m	Lighthouse/state-estimate position used for path visualization and zone mapping.

8.5 Data Quality Checks

- Confirm each CSV includes datetime_et and Lighthouse X/Y/Z columns before using flight-path or zone-map features.
- Compare raw_temp and corr_temp to verify the correction engine is behaving realistically.
- Check battery voltage for sudden drops that may explain unstable data or shortened missions.
- If a run looks abnormal, use the Mission Log and Flight Launcher Output to see whether the script reported errors.

9. The CropWatcher Front-End Dashboard

9.1 Starting the Dashboard

Start the API server from the project folder. The server hosts mainFrontEnd_final.html, provides live telemetry endpoints, loads organized CSV files, and launches flight scripts.

```
python api_server_final.py
http://127.0.0.1:5001
```

9.2 Modes of Operation

- Live/API mode: api_server_final.py is running and the dashboard can receive telemetry, mission events, and flight script output.
- Historical CSV mode: previously saved CSV files are loaded for review, graphing, mapping, and comparison.
- Manual CSV upload mode: the operator can upload a CSV directly into the dashboard for inspection.

9.3 Tab Reference

Tab	Purpose
-----	---------

Dashboard	High-level overview with data source, zone count, average temperature, pressure, alerts, and telemetry charts.
Zone Health Map	Grid-based spatial view of crop health, temperature, or pressure metrics by zone.
Time Series	Plots selected numeric CSV columns over time.
Flight Path	Plots the drone path using Lighthouse X/Y columns such as <code>lh_x_m</code> and <code>lh_y_m</code> .
Flight Control	Launches <code>waypoint_flight.py</code> , <code>hover_test.py</code> , or <code>lawnmower_flight.py</code> from the browser and displays live script output.
Stress Forecast	Displays model-based risk estimates using available telemetry and optional canopy/LAI fields.
Raw Data Table	Shows every loaded CSV row and column for detailed inspection.
Run Comparison	Compares multiple runs across temperature, pressure, altitude, battery, thrust, and other metrics.
Live Telemetry	Shows real-time corrected temperature, pressure, altitude, battery, state, mode, event, and related metrics.
Camera Feed	Diagnostic/optional camera view. Use only if AI Deck camera streaming is confirmed.
Alerts	Shows stress alerts, safety warnings, geofence issues, and system events.
Layout Editor	Defines greenhouse zones and waypoint paths for dashboard mission execution.
Mission Log	Records flight, data, safety, and mission events.
Settings	Contains dashboard settings such as grid/geofence parameters.

9.4 Flight Control Tab

The Flight Control tab is the main browser-based entry point for launching approved scripts. It shows whether the flight API is online, whether a script is running, and provides a Stop Script button for aborting an active process.

- **Crazyflie URI:** Radio address used by the flight script.
- **Hover altitude:** Height used by `hover_test.py`.
- **Ambient temp:** Value passed to `lawnmower` and `waypoint` scripts for sensor correction/logging.
- **Waypoint Planner:** Launches `waypoint_flight.py`.
- **Hover Test:** Launches `hover_test.py`.
- **Lawnmower Flight:** Launches `lawnmower_flight.py`.
- **Flight Launcher Output:** Shows live stdout/stderr from the active script.

9.5 Layout Editor and Mission Log

The Layout Editor is where waypoint missions are created from greenhouse zones. The Mission Log helps the operator review what the dashboard did, including waypoint additions/removals, exports, API events, and mission execution status.

10. Troubleshooting

10.1 Connection and Driver Issues

Problem	Fix
Radio not found in cfclient	Confirm Crazyradio PA is plugged in. On Windows, check Zadig/libusb-win32. On macOS, check USB permissions and try another USB port or adapter.
API Disconnected in dashboard	Start <code>api_server_final.py</code> in the project folder and refresh the browser. If port 5001 is already in use, close the other process or terminal.
Flight Control Offline	Confirm <code>api_server_final.py</code> is running and the script files are in the same folder as the API server.
Connection drops during flight	Move the host closer to the flight area, reduce interference, and confirm the Crazyradio URI/channel is correct.

10.2 Flight Stability and Positioning

Problem	Likely Cause	Fix
Drone drifts or crosses boundaries	Lighthouse geometry or estimator issue.	Repeat geometry estimation, reset estimator, verify X/Y/Z in cfclient, and run <code>hover_test.py</code> .
Lighthouse tab stays red	Base stations blocked, wrong channels, dirty sensors, or poor line of sight.	Check channels, clean sensors, remove obstructions, and re-aim base stations.
Tumble on takeoff	Wrong prop orientation or damaged prop/motor arm.	Verify CW/CCW prop locations and inspect arms/motors.
Altitude seems wrong	Barometer baseline/pressure estimate may be off or drone moved during initialization.	Keep drone still during startup, check pressure readings, and rerun the script.

10.3 Data and Dashboard Issues

Problem	Fix
No latest CSV appears	Check <code>organized_flights/YYYY-MM-DD/</code> for timestamped CSV files. Confirm the script had permission to write files.
Flight Path tab says no position data	CSV must include <code>lh_x_m</code> and <code>lh_y_m</code> . Use updated <code>lawnmower_flight.py</code> or <code>waypoint_flight.py</code> with state logging enabled.
Temperature units look wrong	The launch prompt controls stored units. Enter 74F for Fahrenheit columns or 22C for Celsius columns.
Charts look flat or repetitive	Check whether the drone actually moved through different zones, whether sensor values changed meaningfully, and whether downsampling is hiding small changes. Use the Raw Data Table for

exact values.

10.4 LED Reference

Pattern	Meaning	Action
Solid red on M1	System error	Restart the drone and check expansion decks.
Flashing red on M1	Low battery	Charge or swap the LiPo battery.
Flashing blue on M4	Bootloader mode	Flash firmware or tap power once to exit if flashing is not intended.
Pulsing green	Ready/normal operation	Proceed only after the pre-flight checklist is complete.

11. Maintenance and Support

11.1 Physical Maintenance

- Clean camera/optical surfaces with a soft swab if camera diagnostics are being used.
- Inspect propellers before every mission and replace any damaged blade immediately.
- Inspect motor arms after hard landings or about every 20 flight minutes.
- Keep the Lighthouse deck sensors clean and unobstructed.
- Keep battery wires tucked away from propellers.

11.2 Battery Care

- Do not leave batteries deeply discharged overnight.
- Do not store LiPo batteries fully charged for long periods. Use an appropriate storage charge when the system will not be used for more than two weeks.
- Let batteries cool before charging after heavy use.
- Retire swollen, punctured, or overheated batteries.

11.3 Software Maintenance

- Keep a known-good copy of the final project repository before making experimental changes.
- Use a virtual environment so dependency changes do not break other Python projects.
- Periodically export or back up `organized_flights` and `mission_paths` folders if flight history must be preserved.
- Only update firmware or dependencies when you have time to retest hover, lawnmower, and waypoint missions afterward.

11.4 Component Replacement

- Propellers: Pull the old propeller straight up and install the correct CW or CCW replacement.
- Motors: Disconnect the white motor plug, slide the motor out of the arm, insert the new motor, and route the wire cleanly along the arm channel.
- Motor arms: Replace cracked or bent arms before flying again. Even small cracks can cause vibration and unstable control.

12. Quick Reference

12.1 Common Commands

Task	Command
Activate venv on macOS	source venv/bin/activate
Activate venv on Windows	.\venv\Scripts\activate
Start cfclient	cfclient
Start dashboard/API	python api_server_final.py
Open dashboard	http://127.0.0.1:5001
Run hover test	python hover_test.py
Run lawnmower scan	python lawnmower_flight.py
Run waypoint planner	python waypoint_flight.py

12.2 Files and Folders

File / Folder	Purpose
mainFrontEnd_final.html	CropWatcher browser dashboard.
api_server_final.py	Local API server, telemetry endpoint, CSV loader, and flight script launcher.
hover_test.py	Position-hold and takeoff/landing validation script.
lawnmower_flight.py	Serpentine scan mission with sensor logging and API publishing.
waypoint_flight.py	Custom waypoint planner and waypoint mission executor.
organized_flights/	Date-based CSV output folder.
mission_paths/	Saved dashboard waypoint mission files.
drone_mapping_data.csv	Legacy fallback CSV filename if organized flight logs are not present.